

INTEGRACIÓN FRONT - BACK

Clase #19 - 7 de septiembre de 2026

Matías Corti - Martina García Améndola



DÓNDE QUEDAMOS

La clase pasada cerramos con esta deuda. Qué pasa con ella hoy:

Pendiente de C18	Lo resuelve
Front sigue leyendo libros.json	← HOY · fetch a la API real
No hay pantalla de login	← HOY · login mínimo + token
El token queda suelto en localStorage	Próxima Clase · sesión y contexto
No hay AuthContext ni rutas protegidas en React	Próxima Clase · autorización en el front
Los botones se ven aunque no puedas usarlos	Próxima Clase · renderizado por rol
CORS con dominio real y HTTPS	Clase de deploy

TEMAS DE HOY

- **Origen** y por qué `5173` y `3000` son dos mundos distintos
- **CORS**: qué es, por qué existe, y quién lo aplica
- **Variables de entorno** de los dos lados, y por qué las del front no son secretas
- **El contrato**: lo que el back manda no es lo que el front asumía
- **apiFetch**: una sola puerta de salida al backend
- **Login mínimo**: formulario + `POST /api/auth/login` + el token en el header
- **Debugging de integración**: el árbol de decisión
- **Práctica**: `mock` → API real → CORS → contrato → token



REPOSITORIO $\neq$
SERVICIO $\neq$
SERVIDOR

Repositorio (proyecto)

- Git versiona archivos. Puede tener varios proyectos adentro (front, back, documentación)
- No ejecuta nada. Un `git pull` no levanta un servidor

Servicio

- Un proceso en ejecución que ofrece una funcionalidad
- El backend de Express (`3000`), el frontend de Vite (`5173`), PostgreSQL (`5432`)
- Se levantan, se reinician y se rompen **por separado**

Servidor (host)

- La máquina o el entorno donde corren los servicios
 - Hoy es tu computadora (`localhost`). Luego será un servidor en la nube
 - **Un container es como un host aislado adentro de tu máquina**
-



UN ORIGEN =
PROTOCOLO +
HOST + PUERTO

¿Qué es un origen? Las tres cosas juntas: **protocolo + host + puerto**.
Si cambia **una sola**, ya es otro origen.

URL	¿Mismo origen que http://localhost:5173 ?
<code>http://localhost:5173/catalogo</code>	✅ sí (la ruta no cuenta)
<code>http://localhost:3000</code>	❌ no (otro puerto)
<code>https://localhost:5173</code>	❌ no (otro protocolo)
<code>http://127.0.0.1:5173</code>	❌ no (otro host, aunque sea la misma máquina)
<code>http://api.libreria.com</code>	❌ no (todo distinto)

Nuestro caso: front en `http://localhost:5173`, API en `http://localhost:3000`.

Mismo host, distinto puerto → dos orígenes.



Cross-Origin Resource Sharing

¿Qué es?

- Un mecanismo **de los navegadores** que permite o bloquea pedidos entre orígenes distintos
- El servidor declara qué orígenes acepta; el navegador hace cumplir la declaración

¿Por qué existe? La *same-origin policy*

- Por defecto el navegador impide que una página lea la respuesta de otro origen
- Sin esa regla: entrás a **sitio-cualquiera.com**, ese sitio le pega a **tu-banco.com** con tus cookies y **lee la respuesta**
- La regla no impide que el pedido salga. Impide que la página **lea la respuesta**

Nuestro caso, en la consola del navegador:

```
Access to fetch at 'http://localhost:3000/api/libros' from origin  
'http://localhost:5173' has been blocked by CORS policy: No  
'Access-Control-Allow-Origin' header is present on the requested resource.
```



LO APLICA EL NAVEGADOR

Quien bloquea es el navegador. El servidor contesta igual.

Corolario 1: por qué nunca lo vieron

`api.http`, Postman y `curl` **no son navegadores:** no aplican CORS.

Todo C14-C18 probaron la API sin navegador. Por eso hoy es la primera vez que aparece.

Corolario 2: CORS no protege al servidor

CORS protege **al usuario del navegador**, no a tu API.

Cualquiera con `curl` le pega a tu endpoint sin que CORS diga nada.

```
# CORS no lo frena. Lo que lo frena es el 401 de C18.  
curl -X DELETE http://localhost:3000/api/libros/1
```

Lo que protege al servidor es **autenticación y autorización**, no CORS.



EL MISMO REQUEST, MIRADO DE LOS DOS LADOS

Sin el middleware `cors`, el mismo request se ve **distinto** según dónde mires:

Dónde mirás	Qué ves
Consola del navegador	✗ blocked by CORS policy
Pestaña Network	El request está, con status 200
Log del container de la API	Nada raro. Respondió 200
Base de datos	Si era un DELETE, el dato ya se borró

Respuesta real del backend sin `cors` (verificado):

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 2588
    ← no hay ningún header Access-Control-*
```

El servidor hizo todo el trabajo. El navegador escondió la respuesta.



EL PREFLIGHT

Para pedidos "no simples", el navegador **pregunta antes de mandar**. Ese pedido previo es el preflight: un **OPTIONS**.

¿Qué lo dispara?

- Un método que no sea **GET**, **HEAD** o **POST**
- Un **Content-Type: application/json**
- Un header propio, como **Authorization** ← nuestro caso desde C18

Qué pregunta el navegador y qué contesta el server
(verificado):

```
OPTIONS /api/libros
Origin: http://localhost:5173
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type,authorization
```

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
Access-Control-Allow-Headers: content-type,authorization
```

Recién si el **OPTIONS** pasa, el navegador manda el **POST** de verdad. **En Network vas a ver dos requests.**



`CORS` EN EXPRESS 5

```
// backend/src/index.ts
import cors from "cors";

const app = express();

const corsOptions = {
  origin: [process.env.FRONTEND_URL ?? "http://localhost:5173"],
};

app.use(cors(corsOptions)); // ← 1º, ANTES de json() y de las rutas
app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/libros", libroRoutes);
```

El orden no es estético. Con `cors` **después** del router, no aparece ningún header. Falla completa, no parcial.

No hace falta `app.options('*', cors())`. Con el `app.use` global el preflight se resuelve solo (y en Express 5 esa línea **crashea el arranque**):

```
PathError: Missing parameter name at index 1: *
```



CONFIG POR ENTORNO, Y LO QUE NO VA

El origen permitido **cambia con el entorno**, así que no se escribe a mano:

```
# backend/.env  
FRONTEND_URL=http://localhost:5173
```

Qué pasa según cómo configures **origin**:

origin	Origen no permitido responde
["http://localhost:5173"] (usamos esto)	200 con body, sin Access-Control-Allow-Origin → el navegador bloquea
"http://localhost:5173" (string)	200 con el header del origen configurado, no el pedido → el navegador compara y bloquea
"*"	Cualquiera puede leer tu API desde cualquier página.

Lo que **NO** ponemos: **credentials: true.**

Sirve para que viajen cookies entre orígenes. Nuestro token va en el header **Authorization**, no en cookie. Además es incompatible con **origin: "*" (y el server no te avisa: manda los dos headers y el que rechaza es el navegador).**



MINI-CIERRE: ORÍGENES Y CORS

Concepto	En una línea
Origen	protocolo + host + puerto. Cambia uno, es otro origen
Same-origin policy	El navegador no deja que una página lea la respuesta de otro origen
Quién bloquea	El navegador. El servidor responde igual, y el efecto ya pasó
Por qué no lo vieron antes	api.http, Postman y curl no son navegadores
CORS no es seguridad del server	Protege al usuario. Al server lo protegen 401 y 403
Preflight	OPTIONS previo cuando hay método raro, JSON o Authorization
Implementación	<code>app.use(cors(corsOptions))</code> antes de las rutas
Config	origin como lista, desde <code>FRONTEND_URL</code> . Sin credentials, sin ruta options



VARIABLES DE ENTORNO, Y EL `.ENV.EXAMPL E`

¿Qué es? Un valor que vive **afuera** del código y cambia según dónde corra la app.

¿Por qué? El mismo código tiene que funcionar en tu máquina, en la de tu compañero y en producción, sin editar archivos.

Archivo	¿Va a Git?	Qué tiene
.env	✗ nunca	Los valores reales de tu máquina
.env.example	✓ sí	Las mismas claves, con valores de mentira

```
# backend/.env.example
JWT_SECRET=cambiame por una cadena_larga_y_aleatoria
FRONTEND_URL=http://localhost:5173
```

El `.env.example` es la **documentación de qué variables hay que definir**. Sin él, el que clona el repo no tiene idea de qué le falta y solo ve el server explotar.

⚡ EL `.ENV` DEL FRONT: PREFIJO `VITE\_`

Vite **solo** expone al código las variables que empiezan con **VITE_**.

```
# frontend/.env  
VITE_API_URL=http://localhost:3000/api
```

```
const BASE = import.meta.env.VITE_API_URL;    // "http://localhost:3000/api"
```

Sin el prefijo, queda **undefined** (y el fetch arma una URL absurda):

```
// API URL=http://localhost:3000/api ← sin VITE  
fetch(`${import.meta.env.API_URL}/libros`)    // → "undefined/libros"
```

Para que TypeScript no lo trate como **any**:

```
// frontend/src/vite-env.d.ts  
/// <reference types="vite/client" />  
interface ImportMetaEnv { readonly VITE_API_URL: string }  
interface ImportMeta { readonly env: ImportMetaEnv }
```

⚠ Esto tipa lo declarado, pero **no detecta el tipo**: **VITE_API_UR**
compila igual.



LAS VARIABLES DEL FRONT NO SON SECRETAS

En el build, Vite **reemplaza la variable por su valor literal** dentro del bundle:

```
npm run build
grep -r "localhost:3000" dist/
# dist/assets/index-D4f9a.js:  var BASE = "http://localhost:3000/api";
```

Ese archivo se lo descarga **todo el que entra a tu sitio.**

	Backend	Frontend
Dónde corre	En el servidor	En el navegador del usuario
¿Quién ve las variables?	Solo el servidor	Cualquiera, con F12
Qué puede ir	JWT_SECRET, password de la DB	URLs, flags, IDs públicos
Qué nunca	-	Claves de API, secretos, passwords

Regla: si te importa que nadie lo lea, no puede estar en el front.
Punto.

Si una operación necesita un secreto, **la hace el backend.**



MINI-CIERRE: VARIABLES DE ENTORNO

Concepto	En una línea
Para qué	El mismo código en distintos entornos, sin editar archivos
<code>.env / .env.example</code>	El real fuera de Git; el de ejemplo adentro, con placeholders
Prefijo del front	Sin <code>VITE_</code> , la variable queda undefined
Tipado	<code>src/vite-env.d.ts</code> con <code>ImportMetaEnv</code> . Tipa, no valida el typo
Back vs front	Las del back son secretas. Las del front, no: quedan en el bundle
Regla	Si necesita un secreto, la operación la hace el backend
Al cambiar el <code>.env</code>	Vite se reinicia solo; la pestaña abierta hay que recargarla



EL CONTRATO ACOPLA LOS DOS LADOS

El contrato de la API es lo único que mantiene unidos dos programas separados:

URL + método + forma del body + status codes

Si un lado lo cambia sin avisar, el otro se rompe. Y se rompe en runtime, no al compilar.

El contrato real de la Librería:

```
POST /api/auth/login
  → 200 { "token": "eyJ...", "usuario": { "id": 1, "email":
        "admin@libreria.test", "nombre": "Admin", "rol": "ADMIN"
  }}

GET /api/libros
  → 200 (público, sin header)

GET /api/autores
  → 200 (público, sin header)

POST /api/libros    Authorization: Bearer eyJ...
  → 201 | sin header: 401 | rol CLIENTE: 403 | body inválido: 400
```



SERIALIZACIÓN JSON: NO HAY TIPOS RICOS

Entre el back y el front hay **JSON** que solo tienen: string, number, boolean, null, array, object.

Todo lo demás **se aplana a string** en el camino.

En el back	En el JSON	Lo que asume el front
DateTime de Prisma	"2026-09-07T14:30:00.000Z" (string)	Date → fecha.getFullYear() explota
Decimal de Prisma	"4500.00" (string)	number → precio.toFixed(2) explota
Int de Prisma	4500 (number)	✅ coincide
Relación con include	objeto anidado	depende de cómo lo pediste

En la Librería **precio** es **Int** y no hay ningún **DateTime** en las respuestas, así que **hoy no nos pega**.

En su TP, sí. Si tienen fechas o plata con decimales, este es el bug que van a tener.

Y otra: **res.json()** **puede explotar**. Si el body no es JSON, tira **SyntaxError**.



MOCK VS API REAL: LA TABLA DE DAÑOS

Lo que devuelve `GET /api/libros` de verdad:

```
{ "id": 1, "titulo": "El principito", "autorId": 1, "precio": 4500,  
  "imagen": "/img/principito.jpg", "disponible": true,  
  "autor": { "id": 1, "nombre": "Antoine de Saint-Exupéry", "nacionalidad": "Francia"  
} }
```

Campo	libros.json	API real	Qué pasa
autor	"Antoine de..." string	objeto { id, nombre, nacionalidad }	💣 React explota
autorId	no existe	1	El alta necesita el id, no el nombre
categorias	no existe	solo en GET /api/libros/:id	Lista y detalle no tienen la misma forma
precio	4500	4500	✓
disponible	true	true	✓

El error, textual:

```
Objects are not valid as a React child  
(found: object with keys {id, nombre, nacionalidad}).
```



`APIFETCH`: UNA SOLA PUERTA DE SALIDA

El `useFetch` de C11 tiene tres problemas ahora que hay una API real:

- La URL está a mano
- **Pierde el mensaje de error de la API**
- No manda el token

```
// frontend/src/services/api.ts
const BASE = import.meta.env.VITE_API_URL;

export async function apiFetch<T>(ruta: string, opciones: RequestInit = {}):
Promise<T> {
  const token = obtenerToken();
  const res = await fetch(`${BASE}${ruta}`, {
    ...opciones,
    headers: { 'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...opciones.headers },
  });
  const cuerpo = await res.json().catch(() => null); // el 404 de ruta viene en
  HTML
  if (!res.ok) throw new Error(cuerpo?.error ?? `Error ${res.status}`);
  return cuerpo as T;
}
```

Un solo lugar donde se define la base URL, se manda el token y se leen los errores.



MINI-CIERRE: EL CONTRATO

Concepto	En una línea
El contrato	URL + método + shape + status. Lo único compartido entre los dos lados
Se rompe en runtime	TypeScript no lo puede chequear: el JSON llega de afuera
Serialización	JSON no tiene tipos ricos. Fechas y decimales viajan como string
Dónde se arregla	Lo que es del back, en el back. Nunca parchear en cada componente
Consistencia	La lista y el detalle del mismo recurso tienen que tener la misma forma
apiFetch	Una sola puerta: base URL, token, y el mensaje real del error
El mock	Una hipótesis sobre el contrato. Se verifica contra la API real



ÁRBOL DE DECISIÓN DE DEBUGGING

Dos ventanas abiertas siempre: consola del navegador + log del container.

Síntoma	Dónde mirar	Causa típica
No aparece nada en Network	El código del front	El fetch no se ejecuta
Failed to fetch / status 0	docker compose ps	La API está caída o el puerto está mal
CORS en consola, 200 en el log del server	Config de cors del back	Falta el middleware, u origen mal
404	La URL completa en Network	Base URL, prefijo /api, typo
200 con Content-Type: text/html	La URL en Network	URL relativa: te contestó el dev server, no la API
400	El body que mandás	Zod rechazó el input. El detalle está en la respuesta
401	El header Authorization	Sin token, mal armado o expirado
403	El rol del usuario	Autenticado, pero sin permiso
500	Log del container, no el navegador	Error del backend
200 pero la UI vacía o rota	El JSON real vs el tipo del front	El contrato es distinto



LO QUE FALTA, FALTA A PROPÓSITO

Front conecta con la API real y sabe mandar un token. Lo que **no** hacemos hoy:

Pendiente	Qué es	Por qué hoy no
AuthContext	La sesión como estado global	Es un tema de React, no de integración. (Próxima Clase)
PrivateRoute	Rutas que exigen estar logueado	Idem. (Próxima Clase)
Esconder botones según el rol	UI condicional por permiso	(Próxima Clase)
Matriz de permisos en el front	La tabla de C18 aplicada a la UI	(Próxima Clase)
Cookies httpOnly vs localStorage	Dónde guardar el token	Hoy: localStorage + deuda declarada. (Próxima Clase)
HTTPS y CORS con dominio real	El origen deja de ser localhost	(Clase de deploy)
Testing de la integración		(Clase de testing)
Refresh tokens, rate limiting, 2FA		Fuera del alcance de la materia

Hoy el token vive en **localStorage** y se lee con dos funciones sueltas. En la próxima clase eso se envuelve en un contexto.



PRÁCTICA



PRÁCTICA DEL MOCK A LA API REAL

Ejemplo Completo:
DS26-Libreria (C19)

Cinco pasos. Cada uno rompe algo y lo arregla.

#	Paso	Qué rompe	Min
1	Preparar el terreno	-	8
2	El choque con CORS	El navegador te bloquea	18
3	El contrato de datos	La UI explota con datos correctos	15
4	La puerta de salida (apiFetch)	El 404 que no es JSON	12
5	La capa de sesión	401 · 403 · 201	20



PASO 1

PREPARAR EL

TERRENO

[Ejemplo Completo:](#)
[DS26-Libreria \(C19\)](#)

```
# 1) Base y API arriba
docker compose ps                                # api y db arriba, db "healthy"

# 2) El cliente de Prisma, generado (NO está en Git)
docker compose exec api npx prisma generate

# 3) La dependencia de hoy, DENTRO de la imagen
docker compose exec api npm ls cors              # → cors@2.8.6

# 4) El front, en el host
cd frontend && npm run dev                        # http://localhost:5173
```

Listo cuando: los dos containers arriba, `npm ls cors` muestra una versión, y el catálogo abre en el navegador (todavía leyendo el mock).

⚠ Si te pasa esto	Es porque
<code>npm ls cors</code> dice (empty)	Instalaste cors pero no rehiciste la imagen → <code>docker compose up -d --build</code>
Property 'usuario' does not exist on type 'PrismaClient'	Te saltaste el paso 2. <code>src/generated/prisma</code> está en el <code>.gitignore</code>
No instalaste cors y no hay internet	Plan B: descomentá el proxy en <code>vite.config.ts</code> . Cero instalación
Tu backend de C18 está incompleto	Plan C: de a dos con un compañero, o corré el branch C18 del ejemplo



PASO 2

EL CHOQUE CON CORS

[Ejemplo Completo:](#)
[DS26-Libreria \(C19\)](#)

Archivos a crear / modificar

Archivo	Qué
frontend/.env	VITE_API_URL=http://localhost:3000/api · no va a Git
frontend/.env.example	La misma clave · sí va a Git
frontend/src/vite-env.d.ts	El ImportMetaEnv de la filmina 14 (nuevo, el repo no lo tenía)
frontend/src/pages/LibrosCatalogo.tsx	useFetch<LibroCardProps[]>('/libros') · antes '/libros.json'

Guardá y mirá. Se rompe, y hay que mirarlo en tres lugares:

Dónde	Qué ves
Consola	blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
Network	El request está. Status 200
docker compose logs -f api	Nada raro. Respondió 200



PASO 2

LA SOLUCIÓN

[Ejemplo Completo:](#)
[DS26-Libreria \(C19\)](#)

En `backend/src/index.ts`:

```
import cors from "cors";  
const corsOptions = { origin: [process.env.FRONTEND_URL ?? "http://localhost:5173" ]  
};  
app.use(cors(corsOptions)); // ← ANTES de express.json() y de las rutas
```

`docker compose restart api` → recargá → **el catálogo trae datos.**

⚠ Si te pasa esto	Es porque
undefined/libros y después SyntaxError: Unexpected token '<'	A la variable le falta el prefijo VITE_. Es un 200 con HTML del dev server
Cambiaste el .env y sigue el valor viejo	Vite se reinició solo. Recargá la pestaña
Pusiste cors después de las rutas	Cero headers. Falla total, no parcial
PathError: Missing parameter name at index 1: *	Pegaste <code>app.options('*', cors())</code> . En Express 5 eso no arranca. Borralo: no hace falta
Failed to fetch en vez del error de CORS	No es CORS. La API está caída. <code>docker compose ps</code>



PASO 3

EL CONTRATO DE DATOS

[Ejemplo Completo:](#)
[DS26-Libreria \(C19\)](#)

CORS ya nos deja pasar. Ahora **llegan los datos correctos y la UI explota:**

```
Objects are not valid as a React child  
(found: object with keys {id, nombre, nacionalidad}).
```

El mock aplanaba el autor a un string. La API real manda el objeto de la relación. **Archivos a modificar:**

Archivo	Qué
frontend/src/types/libroCardProps.ts	autor: string → autor: Autor ({ id, nombre, nacionalidad })
frontend/src/components/LibroCard.tsx	{autor} → {autor.nombre}
frontend/src/pages/LibrosCatalogo.tsx	el filtro: l.autor.nombre.toLowerCase()
backend/src/services/libro.services.ts	include: { autor: true } también en create y update
frontend/public/libros.json	git rm - el mock se va

Cambiá el tipo primero y corré:

```
npx tsc -p tsconfig.app.json --noEmit
```

```
LibrosCatalogo.tsx(21,61): error TS2339: Property 'toLowerCase' does not exist on type 'Autor'.  
LibroNuevo.tsx(25,7): error TS2322: Type 'string' is not assignable to type 'Autor'.
```

Un tipo, tres archivos, la lista completa. Para eso se tipa.

¿Quién se adapta? El front al contrato: el mock era la simplificación, no un bug del back.

¿Qué sí se arregla en el back? Que **GET** devuelva el autor anidado y **POST** no. Eso es un defecto del contrato.



PASO 4 · LA PUERTA DE SALIDA

[Ejemplo Completo:
DS26-Libreria \(C19\)](#)

Archivos a crear / modificar

Archivo	Qué
frontend/src/services/sesion.ts	guardarToken / obtenerToken / borrarToken (3 líneas)
frontend/src/services/api.ts	apiFetch: base URL + token + el error real
frontend/src/hooks/useFetch.ts	usa apiFetch adentro. El contrato del hook no cambia
backend/src/index.ts	404 en JSON, después de las rutas y antes del errorHandler

```
// frontend/src/services/api.ts
const cuerpo = await res.json().catch(() => null); // el 404 de ruta viene en HTML
if (!res.ok) throw new Error(cuerpo?.error ?? `Error ${res.status}`);
return cuerpo as T;
```

```
// frontend/src/hooks/useFetch.ts - se van estas tres líneas
const res = await fetch(url);
if (!res.ok) throw new Error('Error al cargar los datos' ); // ← acá se perdía el
mensaje
setData(await res.json());
// y queda una:
setData(await apiFetch<T>(url));
```

⚠ Si te pasa esto	Es porque
Unexpected token '<', "<!DOCTYPE "... is not valid JSON	Typo en la URL. El 404 de ruta viene en HTML
Error 200 (raro)	Idem, pero con el .catch ya puesto: URL relativa, contestó el dev server
El token no viaja después del login	Lo estás leyendo al importar el módulo. Se lee DENTRO de apiFetch



PASO 5 · LA CAPA DE SESIÓN

Ejemplo Completo:
DS26-Libreria (C19)

Nada nuevo: es el formulario de C10 con otro schema y otro submit.

Archivo	Qué
frontend/src/schemas/loginSchema.ts	z.email() + password min(1). El login NO valida fortaleza
frontend/src/pages/Login.tsx	el onSubmit: apiFetch → guardarToken → navigate
frontend/src/pages/LibroNuevo.tsx	el onSubmit: POST /libros. El token lo manda apiFetch solo

```
const session = await apiFetch<Sesion>('/auth/login',  
  { method: 'POST', body: JSON.stringify(datos) });  
guardarToken(session.token);
```

Seed: `admin@libreria.test / Admin1234` · `cliente@libreria.test / Cliente1234`

Probá la matriz de C18, ahora desde el navegador:

Situación	Status	Lo que ves en la UI
Sin loguearte	401	Falta el token
Logueado como CLIENTE	403	No tenés permiso para esta operación
Logueado como ADMIN	201	Vuelve al catálogo con el libro nuevo

Y ahora sí: en Network aparecen **DOS requests**. El header `Authorization` dispara el preflight `OPTIONS (204)`, y recién después va el `POST`.

Si llegás: `logout = borrarToken()`. Probá entrar a `/libros/nuevo` **sin loguearte:** llenás todo el formulario y recién ahí te come un 401. Y logueado como CLIENTE, el 403. La UI no sabe quién sos ni qué podés hacer. **Lo vemos en la próxima clase.**



ROMPELO A PROPÓSITO

Nueve fallas con su mensaje real. **Elegí tres y reproducilas.**

#	Qué hacés	Qué ves
1	Saco <code>app.use(cors(...))</code>	blocked by CORS policy en consola, 200 en el log del server
2	<code>origin: ["http://localhost:5174"]</code>	Sigue bloqueado: 200 con body, sin Access-Control-Allow-Origin
3	Muevo cors después de las rutas	Cero headers. Falla completa
4	Agrego <code>app.options('*', cors())</code>	PathError: Missing parameter name at index 1: * (no arranca)
5	Le saco el prefijo VITE_	200 con text/html, después SyntaxError: Unexpected token '<'
6	Edito el .env y no recargo la pestaña	Vite se reinicia, la página sigue con el valor viejo
7	POST con token de CLIENTE	403 {"error":"No tenés permiso para esta operación"}
8	<code>docker compose stop api</code> y recargo	Failed to fetch. No es CORS: no hay nadie del otro lado
9	<code>JWT_EXPIRES_IN = "10s"</code> , espero, vuelvo	401 {"error":"Token expirado"}, distinto de "Token inválido"



MINI-CIERRE: LA PRÁCTICA

Paso	En una línea
1 · Terreno	<code>prisma generate + cors</code> dentro de la imagen. Diagnóstico antes de tocar
2 · CORS	El request llegó, el server contestó 200, el navegador escondió la respuesta
2 · La solución	<code>cors</code> con origin como lista, desde <code>FRONTEND_URL</code> , antes de las rutas
3 · El contrato	autor era objeto. El front se adaptó, el back se hizo consistente
3 · El mock	<code>libros.json</code> borrado. La promesa de C11, cumplida
4 · <code>apiFetch</code>	Una puerta: base URL, token, y el mensaje real del error
4 · El 404	Un 404 de ruta viene en HTML. <code>res.json()</code> puede explotar
5 · La sesión	Login → token → Authorization. 401 / 403 / 201, y el preflight
Lo que queda	El token suelto en <code>localStorage</code> , sin contexto (Próxima Clase)



PREPARÁNDOSE
PARA LA ENTREGA



ENTREGA BLOQUE 3

13/09/2026



RECORDATORIO

El 13/09/2026 es la
ENTREGA 3 (Backend) del
Trabajo Práctico Grupal

Checklist para la entrega del Backend

Requerido

- ✓ Estructura de proyecto organizada
- ✓ API REST funcionando: al menos 3 CRUDs
- ✓ PostgreSQL por Docker + Prisma, con relaciones y migraciones commiteadas
- ✓ Validación con Zod (body y params), middleware de error
- ✓ Auth: registro, login, `authenticate` y `authorize` según su matriz de permisos
- ✓ Integración básica real con el frontend
- ✓ Pantalla de login funcionando y token viajando en las escrituras
- ✓ `api.http` con los casos de error de sus entidades: 400, 401, 403, 404, 409
- ✓ Documentación de endpoints en la carpeta de Documentación
- ✓ Notas de la entrega en el `README.md` del backend

Fundamental

- ✓ Participación de **todos** los integrantes
- ✓ **Avisar por Discord** en el canal del grupo cuando esté subido



ENTREGA 3 BACKEND CON EXPRESS



RECORDATORIO

El 13/09/2026 es la
ENTREGA 3 (Backend) del
Trabajo Práctico Grupal

Trabajo Práctico, Backend con Express (Grupal)

El checklist previo es una guía para la entrega.

Aplicar al trabajo práctico los temas vistos en las clases y lo que consideren apropiado para su proyecto.

En caso de no poder aplicar todo, al menos completar los que puedan.

Adatar el archivo **README.md** original de la carpeta "backend", dejando en él una explicación de la implementación y los comentarios que crean convenientes para la evaluación.

Recuerden:

- Distribuir el trabajo entre **todos los compañeros** del equipo y apoyar a los que vienen atrasados
 - Continuar aplicando **Pull Requests** y **revisiones cruzadas**
 - **Utilizar tablero Trello** del equipo para el seguimiento
 - **Avisar EN TÉRMINO** por el canal del Grupo de Discord una vez **completada y subida** al repositorio de GitHub
- CHEQUEAR ESTO ENTRE TODOS: completa + subida + aviso**
- La **entrega en término** es fundamental, sin excepción.
 - **TODO EL PROCESO ES PARTE DE LA EVALUACIÓN.**



REVISAR EL REPO ANTES DE ENTREGAR

Estructura correcta

```
/
├── frontend/           → React (5173)
├── backend/            → Express (3000)
├── Documentación/
├── docker-compose.yml
└── README.md
```

❌ frontend/backend/

❌ src/frontend/ + src/backend/

❌ un package.json en la raíz para todo

Lo que NUNCA va a Git

Archivo	Por qué
.env (de los dos)	Secretos
node_modules/	Se reinstala con npm install
dist/	Se regenera con npm run build
backend/src/generated/prisma/	Se regenera con prisma generate



CÓMO SE VE EL TRABAJO DEL EQUIPO

✓ Repo saludable	⚠ Repo deficiente
Commits regulares a lo largo de las semanas	Toda la actividad en los últimos dos días
Todos los integrantes contribuyendo	Dos personas con el 90% de los commits
Pull requests con review cruzado	Push directo a main
Commits chicos y con mensaje claro	Tres commits gigantes tipo "cambios"
README actualizado	README del template de Vite

Trello

- Al día, con responsables asignados
- Los pendientes del último sprint, identificados
- Las tarjetas de esta semana movidas a "Done"

Esto lo vemos todos. Es lo que refleja el trabajo progresivo.

ACTIVIDADES

ACTIVIDAD INDIVIDUAL

Instancia única 13/09/2026



Cierre Bloque 3



También es el límite para las actividades previas del bloque

[AI-C19] Integración en la Librería

Referencia en el repo de ejemplo [branch C19](#)

En `ds2026-actividades`: copiar el sitio de la librería de la Clase 18 en `ejercicios/c19-integracion/`

1. `backend/.env.example` con `FRONTEND_URL`, y `cors` configurado con `origin` como lista (no `"*"`)
2. `frontend/.env` con `VITE_API_URL` (fuera de Git) y `frontend/.env.example` (dentro)
3. `src/vite-env.d.ts` con el `ImportMetaEnv` tipado
4. `services/api.ts` con un `apiFetch` que: arma la URL desde la env, manda el token si existe, y **preserva el mensaje de error de la API**
5. Catálogo leyendo de la API real. `public/libros.json` **borrado**
6. Tipos del front alineados con el JSON real (sin `any`). `npx tsc -p tsconfig.app.json --noEmit` sin errores
7. Pantalla de login con validación (Zod) que guarda el token y muestra el error real del back
8. Un alta que mande el token: verificar `401` sin token, `403` con CLIENTE, `201` con ADMIN
9. Middleware `404` en JSON en el backend
10. Commiteá y avisá en `#actividades`
→ hilo **[AI-C19] Integración en la Librería**

ACTIVIDAD GRUPAL

Hasta el **13/09/2026**
(inclusive)



RECORDATORIO

El **13/09/2026** es la
ENTREGA 3 (Backend) del
Trabajo Práctico Grupal

[Entrega Bloque 3] Frontend conectado al Backend real

Esta actividad es parte de la entrega del Bloque 3

1. `cors` configurado en el backend del TP, con el origen del front por variable de entorno
2. `.env.example` en front y back, con todas las claves y valores de mentira
3. Capa de acceso a la API en el front (un solo lugar con la base URL y el token)
4. **Mocks del front reemplazados por la API real.** Sin `.json` en `public/` que simule datos
5. Tipos del front alineados con el contrato real de su API
6. Pantalla de login funcionando, con los roles de su dominio
7. Al menos una operación de escritura mandando el token, con sus 401 y 403 verificados
8. Repartir el trabajo: al menos **un PR con review cruzado por integrante**
9. Tarjetas de Trello movidas a "Done", retrospectiva del sprint y planning del siguiente
10. Commitear y avisá en el canal del grupo, con parte de la entrega completa del Bloque 3